МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
№ 2

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Управление процессами и обмен данными через каналы
Вариант 4

Выполнил:

студент группы М8О-210БВ-25
Македонский Александр Сергеевич

Москва, 2026 г.

1. Цель работы

Освоить создание и завершение процессов, обмен данными через неименованные каналы и обработку системных ошибок. Разработать на языке C программу из двух отдельных исполняемых файлов: родитель передаёт команды, а потомок выполняет вычисления и сохраняет результаты.

Вариант 4 группы 1: первая строка ввода задаёт имя выходного файла. Каждая следующая строка содержит произвольное количество чисел float. Потомок последовательно делит первое число на остальные: для «100 2 5» результат равен (100 / 2) / 5 = 10. При нулевом делителе проверку выполняет потомок, после чего оба процесса завершаются.

2. Краткое описание архитектуры решения

Решение написано на C11 с использованием POSIX API для Linux. В проекте три исходных файла и общий заголовок: parent.c запускает потомка и передаёт символы, child.c разбирает строки и выполняет деление, common.c содержит операции с байтом, сигналы, закрытие дескрипторов и waitpid. Сборка двух программ задана в src/Makefile.

2.1 Схема взаимодействия компонентов

```
Пользователь → parent
parent → pipe1 → stdin программы child
child → выходной файл с результатами
stdout программы child → pipe2 → parent
parent и child → stderr с PID
```

Рисунок 1 — Поток данных и сообщений между процессами

Имя файла передаётся потомку аргументом execvp. Через dup2 вход потомка связывается с pipe1, а выход — с pipe2. Родитель оставляет запись в pipe1 и чтение pipe2; потомок — чтение pipe1 и запись pipe2. Ненужные концы каналов закрываются. Общая память и дополнительные потоки не используются.

Потомок отправляет два вида однобайтовых сообщений: A — готовность принять следующую строку, Z — остановка при нулевом делителе. Первый A отправляется после открытия выходного файла. После каждого перевода строки parent ждёт ответ; следующая команда после Z не отправляется. Ошибка потомка обнаруживается по закрытию канала и коду waitpid. При EOF родитель закрывает канал команд и ждёт потомка.

3. Листинг ключевых фрагментов кода

Ниже приведены фрагменты текущих исходников. Полный код находится в каталоге src; адрес репозитория:

https://github.com/amakedoni/oci

3.1 Создание каналов и запуск потомка

В parent.c, строки 50–63, создаются два канала и выполняется fork. Дочерняя ветвь перенаправляет стандартные потоки через dup2, закрывает исходные дескрипторы и запускает child через execvp. При отказе execvp потомок завершает работу с кодом 127. Обычный запуск выполняется из корня репозитория; путь к child можно также передать дополнительным аргументом.

```
    if (pipe(commands) == -1) { report_error("Создание канала
команд"); goto cleanup; }
    if (pipe(replies) == -1) { report_error("Создание канала
ответов"); goto cleanup; }
    child = fork();
    if (child == -1) { report_error("Создание потомка"); goto
cleanup; }
    if (child == 0) {
        if (dup2(commands[0], STDIN_FILENO) == -1 ||
            dup2(replies[1], STDOUT_FILENO) == -1)
{ report_error("Перенаправление потоков"); _exit(1); }
        if (close_pipes(commands, replies) == -1) _exit(1);
        char *program = argc == 2 ? argv[1] : "./src/build/child";
        char *arguments[] = {program, filename, NULL};
        execvp(program, arguments);
        report_error("Запуск программы потомка");
        _exit(127);
    }
```

Листинг 1 — Каналы, перенаправление потоков и execvp

Успешный execvp заменяет программу в уже созданном процессе. Он не создаёт ещё один процесс. Родитель после fork продолжает собственную ветвь и проверяет ответ A перед отправкой первой команды.

3.2 Разбор чисел и последовательное деление

В child.c, строки 15–36, функция divide_line читает числа через strtof. Указатель end задаёт границу очередного числа. Проверяются неверные элементы, диапазон и конечность значений. Первое число становится ответом, последующие — делителями; проверка нуля выполняется до деления.

```
static enum calculation divide_line(const char *line, float
*result)
{
    int first = 1;
    while (*line != '\0') {
        while (isspace((unsigned char)*line)) ++line;
        if (*line == '\0') break;
        char *end;
        errno = 0;
        float number = strtof(line, &end);
        if (end == line || (*end != '\0' && !isspace((unsigned
char)*end)))
            return CALC_INVALID;
        if (errno == ERANGE || !isfinite(number)) return
CALC_INVALID;
        if (first) { *result = number; first = 0; }
        else {
            if (number == 0.0f) return CALC_ZERO;
            *result /= number;
            if (!isfinite(*result)) return CALC_INVALID;
        }
        line = end;
    }
    return first ? CALC_EMPTY : CALC_OK;
}
```

Листинг 2 — Вычисление результата одной команды

Сравнение с 0.0f обнаруживает и положительный, и отрицательный ноль. Ноль первым операндом допустим: «0 5» даёт 0. Один операнд записывается без изменения, пустая строка пропускается. Вывод использует FLT_DECIMAL_DIG; вычисления float подвержены округлению, а очень малый промежуточный результат может округлиться до нуля.

3.3 Завершение при делении на ноль

В child.c, строки 61–67, нулевой делитель приводит к отправке Z через второй канал. Потомок завершает цикл с кодом 2. Результат ошибочной команды в файл не записывается.

```
        if (status == CALC_ZERO) {
            if (dprintf(STDERR_FILENO, "[Потомок PID=%ld] деление
на ноль\n",
                        (long)getpid()) < 0)
{ report_error("Потомок: запись сообщения"); goto cleanup; }
            if (write_byte(STDOUT_FILENO, 'Z') == -1)
{ report_error("Потомок: отправка ответа"); goto cleanup; }
            result = EXIT_ZERO_DIVISION;
            break;
        }
```

Листинг 3 — Уведомление родителя о нулевом делителе

В parent.c, строки 99–105, ответ Z прекращает обработку ввода. В cleanup закрываются каналы, вызывается reap_child и освобождается имя выходного файла.

3.4 Чтение строк и управление ресурсами

Для строк используется стандартная POSIX-функция getline: она сама выделяет и увеличивает буфер, поэтому число операндов не ограничено фиксированным массивом. В child.c 44–60 один блок повторно используется до завершения, а затем освобождается через free.

```
    while (!interrupted) {
        ssize_t length = getline(&line, &capacity, stdin);
        if (length == -1) {
            if (feof(stdin)) result = EXIT_SUCCESS;
            else report_error("Потомок: чтение команды");
            break;
        }
```

Листинг 4 — Чтение очередной строки

Родитель передаёт байты через read_byte и write_byte; на переводе строки ждёт ответ. При EOF незаконченная команда дополняется переводом строки. Poll следит за вводом и каналом потомка. Setvbuf отключает предварительное чтение stdin перед сочетанием getline и read.

SIGINT и SIGTERM записываются во флаг interrupted. Poll ждёт до 100 мс, чтобы цикл проверял флаг. SIGPIPE игнорируется: запись без читателя возвращает EPIPE. Закрытие канала команд даёт потомку EOF, waitpid забирает его завершение.

Ошибки проверяются, ресурсы очищаются в cleanup. report_error печатает русское пояснение и номер errno.

4. Результаты работы

Сохранены реальные результаты текущих запусков в report/logs. Локальная среда: macOS 27.2 arm64, Apple Clang 21.0.0, Python 3.9.6. Целевая ОС — Linux/POSIX. Фактическая Linux-трасса strace здесь не получена; ниже приведена команда для её демонстрации.

4.1 Консольный вывод и логи

Обычный запуск выполняется из корня репозитория. Первая строка раздела «ВВОД» содержит имя файла, остальные — команды. «СООБЩЕНИЯ» содержит диагностику, «ВЫХОДНОЙ ФАЙЛ» — результат. Существующий выходной файл при запуске очищается.

```
$ ./src/build/parent < src/examples/normal.in
ВВОД:
src/build/results.txt
100 2 5
7.5 2.5
-24 2 -3
0 5
0.5
СООБЩЕНИЯ:
[Родитель PID=58033] потомок=58034
[Потомок PID=58034] родитель=58033; готов
[Родитель PID=58033] потомок=58034, код=0; завершение получено
Код завершения: 0
ВЫХОДНОЙ ФАЙЛ:
10
3
4
0
0.5
```

PID 58033 относится к parent, PID 58034 — к child. Номер родителя в сообщении потомка совпадает с PID parent. В выходном файле записаны 10, 3, 4, 0 и 0.5. Сообщение «код=0; завершение получено» подтверждает успешный выход и ожидание потомка через waitpid.

Во втором запуске после успешной команды «12 2» поступает строка «8 0». Потомок обнаруживает нулевой делитель и сообщает об остановке; следующая команда «99 3» не обрабатывается.

```
$ ./src/build/parent < src/examples/zero.in
ВВОД:
src/build/zero-result.txt
12 2
8 0
99 3
СООБЩЕНИЯ:
[Родитель PID=58035] потомок=58036
[Потомок PID=58036] родитель=58035; готов
[Потомок PID=58036] деление на ноль
[Родитель PID=58035] потомок=58036, код=2; завершение получено
Код завершения: 2
ВЫХОДНОЙ ФАЙЛ:
6
```

Оба процесса завершаются с кодом 2, в выходном файле остаётся только 6. Следующая команда 99 3 не обрабатывается. PID и сообщение об ожидании подтверждают работу двух процессов. Журналы сохранены в report/logs.

В примере invalid.in первые четыре строки отвергнуты как неверный ввод: неверные элементы, NaN и выход за диапазон float. Диагностика — «неверный ввод; строка пропущена». Следующая команда 9 3 даёт результат 3.

```
[Родитель PID=58037] потомок=58038
[Потомок PID=58038] родитель=58037; готов
[Потомок PID=58038] неверный ввод; строка пропущена
[Потомок PID=58038] неверный ввод; строка пропущена
[Потомок PID=58038] неверный ввод; строка пропущена
[Потомок PID=58038] неверный ввод; строка пропущена
[Родитель PID=58037] потомок=58038, код=0; завершение получено
Код завершения: 0
ВЫХОДНОЙ ФАЙЛ:
3
```

4.2 Проверки краевых случаев

В обычной сборке и сборке с AddressSanitizer и UndefinedBehaviorSanitizer прошли 16 тестов; один тест /dev/full пропущен, поскольку устройство отсутствует на macOS. Диагностики санитайзеров в выполненных сценариях нет. Ниже приведены итоговые строки текущих журналов.

```
$ make -C src test
Ran 17 tests in 0.497s

OK (skipped=1)
$ make -C src sanitize
Ran 17 tests in 1.031s

OK (skipped=1)
```

Проверены дробные и отрицательные числа, ноль первым операндом, делители ±0, неверные элементы, NaN, бесконечность и встроенный NUL. Длинная строка с 100 000 единиц обрабатывается без фиксированного ограничения длины. Результаты 500 команд проверяются с относительным допуском 3e-7. Последняя строка без перевода строки также обрабатывается.

Аварийные сценарии включают пустое имя файла, ошибку открытия, отсутствие программы child со статусом 127, лимит дескрипторов 5, обрыв связи и исчезновение потомка при ожидании ввода. SIGTERM завершает родителя с кодом 143 и ожиданием потомка. При нулевом делителе остановка происходит даже при ещё открытом пользовательском вводе.

Сборка с -Wall -Wextra -Wpedantic -Werror и статический анализ clang --analyze трёх модулей C прошли без замечаний. Журналы сборки, тестов и санитайзеров сохранены в report/logs.

4.3 Демонстрация системных вызовов в Linux

Strace — внешняя утилита Linux, показывающая системные вызовы и сигналы. В исходниках нет функции strace: трассировщик запускается снаружи программы. Для получения журнала на Linux с установленным strace из корня проекта выполняется команда ниже. Это инструкция, а не полученный здесь журнал Linux.

```
strace -f -o src/build/trace.log ./src/build/parent <
src/examples/zero.in
```

Ключ -f включает потомка, -o задаёт файл журнала. Pipe находится в parent.c 50–51, fork — 52, dup2 — 55–56, execvp — 60. Open находится в child.c 43, запись результата через dprintf — 72. Ответ Z отправляется в child.c 64 через write в common.c 38. Poll находится в parent.c 79, waitpid — в common.c 68. На Linux библиотечные функции могут отображаться как pipe2, clone, execve, openat и wait4. Ожидаемые код завершения и содержимое файла: 2 и число 6.

5. Выводы

Реализованы отдельные программы родителя и потомка с двумя каналами, вычислением последовательного деления float и записью результатов в файл. Нулевой делитель проверяется потомком и останавливает оба процесса. Логи с PID, тесты и санитайзеры подтвердили работу на macOS. Освоены создание процессов, перенаправление потоков, обмен через каналы, освобождение памяти и ожидание потомка.